Лабораторная работа № 8
Разработка программ с использованием паттерна *Декоратор*

Цель работы: научиться применять паттерн проектирования *Декоратор* для реализации объекта с дополнительной функциональностью.

1. Теоретические сведения

Задача. Объект, который предполагается использовать, выполняет основные функции. Однако может потребоваться добавить к нему некоторую дополнительную функциональность, которая будет выполняться до, после или даже вместо основной функциональности объекта.

Способ решения. *Декоратор* предусматривает расширение функциональности объекта без определения подклассов.

Участники. Класс *ConcreteComponent* – класс, в который с помощью шаблона Декоратор добавляется новая функциональность. В некоторых случаях базовая функциональность предоставляется классами, производными от класса *ConcreteComponent*. В подобных случаях класс *ConcreteComponent* является уже не конкретным, а абстрактным. Абстрактный класс *Component* определяет интерфейс для использования всех этих классов.

Следствия:

- добавляемая функциональность реализуется в небольших объектах. Преимущество состоит в возможности динамически добавлять эту функциональность до или после основной функциональности объекта *ConcreteComponent*;
- позволяет избегать перегрузки функциональными классами на верхних уровнях иерархии;
- декоратор и его компоненты не являются идентичными.

Реализация. Создаётся абстрактный класс, представляющий как исходный класс, так и новые, добавляемые в класс функции. В классах-декораторах новые функции вызываются в требуемой последовательности – до или после вызова последующего объекта.

При желании остаётся возможность использовать исходный класс (без расширения функциональности), если на его объект сохранилась ссылка.

Замечания и комментарии. Хотя объект-декоратор может добавлять свою функциональность до или после функциональности основного объекта, цепочка создаваемых объектов всегда должна заканчиваться объектом класса *ConcreteComponent*.

И *декоратор*, и *адаптер* являются обёртками вокруг объекта – хранят в себе ссылку на оборачиваемый объект и часто передают в него вызовы методов. Отличие декоратора от адаптера в том, что адаптер имеет внешний интерфейс, отличный от интерфейса оборачиваемого объекта, и используется именно для стыковки разных интерфейсов, в то время как декоратор имеет точно такой же интерфейс и используется для добавления функциональности.

Для расширения функциональности класса возможно использовать как *декораторы*, так и *стратегии*. Декораторы оборачивают объект снаружи, стратегии же вставляются в него внутрь по неким интерфейсам.

Недостаток стратегии: класс должен быть спроектирован с

возможностью добавления стратегий, декоратор же не требует такой поддержки.

Недостаток декоратора: он оборачивает ровно тот же интерфейс, что предназначен для внешнего мира, что вызывает смешение публичного интерфейса и интерфейса кастомизации, которое не всегда желательно.

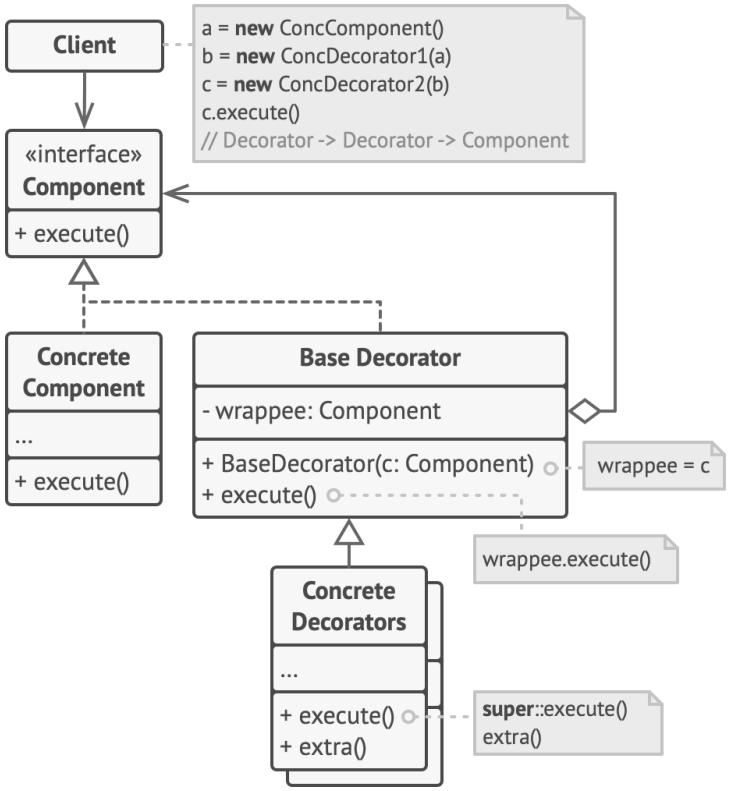


Рисунок 1 – Структура паттерна *Декоратор*

2. Задания к лабораторной работе

Для заданного варианта задания разработать UML-диаграмму классов и диаграмму последовательности.

В качестве основы взять проект, разработанный для лабораторной работы № 7.

С помощью паттерна *Декоратор* декорировать объекты классов-наследников дополнительными свойствами.

Разработать консольное приложение (C++, C#). Допускается вводить дополнительные понятия предметной области. В программе предусмотреть тестирование функциональности созданных объектов классов.

Отчет по лабораторной работе – файл формата pdf. Формат имени файла отчета: <НомерГруппы >_<ФамилияСтудента>.pdf. В отчет включить построенные диаграммы и исходный код программы (при необходимости и заголовочные файлы). Формат отчета см. в Приложении. Отчет загрузить в LMS.

При защите лабораторной работы: уметь объяснить логику и детали работы программы; реализацию паттернов проектирования на примере разработанной программы.

Технологии конструирования программного обеспечения

Отчет по лабораторной работе № 00

Группа: 000-000 **Студент**: Иванов Иван Иванович

Задание на лабораторную работу

<Формулировка задания согласно варианту …… ……..>

Диаграмма классов

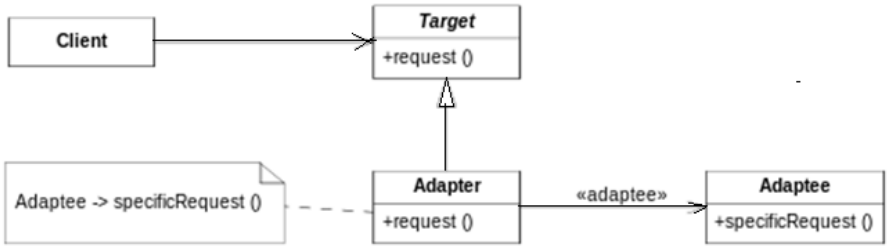
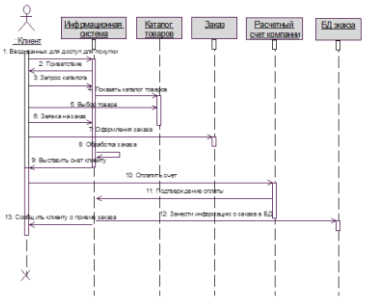


Диаграмма последовательности



Исходный код программы

```
#include "pch.h"
#include "CppUnitTest.h"
#include "../Add/Add.cpp"

using namespace Microsoft::VisualStudio::CppUnitTestFramework;

namespace UnitTest1
{
	TEST_CLASS(UnitTest1)
	{
	public:

		TEST_METHOD(TestMethod1)
		{
			Assert::AreEqual(2, add(1, 1));
		}
```